

Relatório do desempenho e identificação de que cada classe faz e como funciona

CalcDano: colocar a fórmula e as regras do cálculo de dano causado por ataques, considerando tipos elementares, valores limite de status e modificadores

Terreno: controla os efeitos dos campo de batalha como recuperações de vida do terreno canteiro e regras de arenas específicas como EstacionamentoUCSal

Batalha: gerencia o fluxo principal do combate, incluindo o controle da fila/ordem de turnos e a geração de elementos aleatórios

Mochila: guarda todo o inventário do jogador, controlando as regras e o limite de itens utilizáveis permitidos por partida

Inicial: representa a entidade base dos PokéSals e seus atributos base de combate

Ataque: modela as habilidades e golpes disponíveis, especificando poder, tipo elemental e efeitos secundários como alteração de clima ou terreno

Buff: controla os modificadores temporários de atributos ,aumentos e reduções de status , aplicados durante a batalha ativa

Documentação das classes de teste

A suíte automatizada foi estruturada em 5 classes de teste, totalizando 20 testes unitários obtendo 100% de aprovação

CalcDanoTest: valida a precisão dos cálculos de combate. testa multiplicadores de vantagem elemental (testVantagemElemental), limites de entrada (testCalculoDanoBoundaryValues) e comprova o comportamento anômalo do sistema quando a defesa atinge zero (testBugDefZeroCausaDanoAbsurdo)

TerrenoTest: valida as alterações e condições impostas pelo campo de batalha, focando no funcionamento do terreno especial via testEfeitoTerrenoEstacionamentoUCSal()

BatalhaOrdemTest: verifica a mecânica de iniciativa do jogo através do testOrdemDeAtaquePorVelocidade(), garantindo que a ordem de execução do turno respeite os atributos de velocidade.

MochilaTest: avalia a regra de limitação do uso de itens com o teste testUsoLimiteDeltensExcedido(), assegurando que o jogador não exceda a quantidade permitida

RequisitosAutoraisTest: valida as 4 regras de negócio criadas pela equipe:

Dano Crítico: probabilidade de 1/16 para aplicação de dobro de dano

Bônus STAB: aumento de 50% de dano caso o tipo do ataque seja idêntico ao do PokéSal

Escopo de Buffs/Debuffs: garantia de que modificadores de atributo permaneçam ativos apenas durante o combate corrente.

Ataques de Clima/Terreno: Funcionamento de golpes que alteram o estado do ambiente (ex.: Dança da Chuva, Dia Ensolarado).

Análise técnica e conclusão do relatório

O projeto obteve aprovação no Quality Gate do SonarQube, destacando-se pela ausência completa de vulnerabilidades de segurança e zero duplicação de código (0.0% em ~1.1k linhas).

Então, do ponto de vista da avaliação da qualidade do software, existem débitos técnicos e pontos de melhoria prioritários para as próximas fases:

Integração de Cobertura (Coverage em 0.0%): Embora os 20 testes JUnit rodem com sucesso localmente, o sonarqube registrou 0.0% de cobertura em 447 linhas elegíveis. Isso sugere a ausência de um plugin de rastreamento no Maven (como o JaCoCo) para exportar o relatório de execução dos testes para o SonarScanner.

Gargalos de Confiabilidade (Reliability - Nível D): o SonarQube apontou 4 problemas de confiabilidade. O mais severo envolve a instanciação repetida de objetos Random dentro da classe Batalha.java, devendo ser refatorado para reuso do objeto. Houve também alertas de menor gravidade sobre conversão implícita/casting para long no arquivo Inicial.java.

Melhoria de Boas Práticas (84 Code Smells): a quase totalidade dos avisos decorre da utilização direta de System.out.print() para exibição de logs em vez de uma biblioteca de logging estruturada (ex.: SLF4J/Logback).

Correção de Bugs Mapeados: a equipe deve planejar a correção dos 2 erros identificados:

Cura Duplicada: Método terreno.curaTerreno aplicando 10% de cura no terreno CANTEIRO em vez dos 5% especificados.

Estouro por Divisão por Zero: tratar a divisão quando a DEF efetiva for 0 em CalcDano.calcularDano para evitar que a conversão de Infinity gere Integer.MAX_VALUE.

feito por rubens .